

# AI技术应用场景匹配报告

## 一、报告目的与分析依据

随着生成式人工智能、大语言模型和智能体技术的发展，AI 正逐步参与软件工程中的需求分析、系统设计、编码实现、测试验证和运维监控等活动。为了分析 AI 技术对传统软件过程的改进价值，本报告以已完成的《软件过程定义》成果为基础，结合 2024—2026 年 AI 在软件工程领域的研究成果、行业报告和典型工具实践，识别原有软件过程中存在的低效环节，并匹配具有可落地性的 AI 辅助应用场景。

本报告所依据的软件过程覆盖项目启动与计划、需求获取与分析、原型设计与可行性验证、系统架构与详细设计、编码实现与单元验证、集成测试与迭代修正、交付与确认、项目总结与过程改进等主要活动。该过程体现了软件项目从目标确定、需求分析、设计实现、测试验证到交付改进的完整生命周期，也包含相应的角色职责、阶段性交付物和质量控制机制。因此，本报告并不单独讨论 AI 技术本身，而是重点分析 AI 技术如何嵌入既有软件过程活动，辅助改进具体的过程节点和交付物质量。

本报告的分析思路为：首先梳理原有软件过程的阶段划分、主要活动、角色职责、交付物和评审机制；其次，从需求整理、设计建模、代码实现、测试生成和问题复盘等活动中识别效率或质量方面的主要痛点；最后，依据任务重复性、输出可验证性、数据可获得性、效率提升潜力和风险可控性等标准，筛选适合引入 AI 技术的关键场景，形成“AI 辅助 + 人工审核”的应用场景匹配方案。

## 二、原有软件过程概述与痛点分析

### 2.1 原有软件过程阶段概述

| 阶段      | 主要活动                                     | 参与角色                                   | 主要交付物                    | 现有质量控制方式               |
|---------|------------------------------------------|----------------------------------------|--------------------------|------------------------|
| 项目启动与计划 | 分析项目背景，明确项目目标、范围边界、人员分工、进度计划、资源约束和主要风险   | 项目负责人、产品负责人、开发负责人、测试负责人、团队成员、指导教师或客户代表 | 项目计划书、任务分解表、进度计划、风险清单    | 范围评审、计划评审、风险评审、里程碑检查   |
| 需求获取与分析 | 识别用户角色和业务流程，整理功能需求、接口需求、数据需求、非功能需求和需求优先级 | 需求分析人员、产品负责人、用户代表、项目负责人、设计人员、测试人员      | 需求规格说明书、用户故事、用例说明、需求跟踪矩阵 | 需求评审、用户确认、优先级确认、需求追踪检查 |

| 阶段         | 主要活动                                           | 参与角色                                  | 主要交付物                           | 现有质量控制方式                    |
|------------|------------------------------------------------|---------------------------------------|---------------------------------|-----------------------------|
| 原型设计与可行性验证 | 构建界面原型、业务流程原型或技术验证原型，验证核心功能、关键技术和用户交互方式        | 产品负责人、架构设计人员、UI/UX设计人员、开发人员、测试人员、用户代表 | 原型系统、原型评审记录、技术验证结果、可行性结论        | 原型评审、用户反馈、关键技术准入检查、验证结论评审   |
| 系统架构与详细设计  | 进行总体架构设计、模块划分、接口设计、数据库设计、数据结构设计、安全设计、性能设计和部署设计 | 架构设计人员、模块负责人、数据库设计人员、安全负责人、开发人员、测试人员  | 系统设计说明书、UML模型、接口文档、数据库设计文档、部署方案 | 设计评审、接口评审、架构一致性检查、需求到设计追踪   |
| 编码实现与单元验证  | 按设计实现模块代码，完成接口适配、数据处理、业务逻辑实现、单元测试和代码提交         | 前端开发人员、后端开发人员、模块开发人员、测试人员、配置管理人员      | 源代码、单元测试报告、代码提交记录、模块说明          | 版本控制、代码审查、静态检查、单元测试、分支合并准入  |
| 集成测试与迭代修正  | 进行模块集成、接口联调、功能测试、异常测试、性能测试、安全测试和需求覆盖检查         | 测试人员、开发人员、项目负责人、配置管理人员                | 集成测试记录、缺陷记录、测试报告、修正结果           | 测试评审、缺陷闭环、回归测试、覆盖率检查、需求追踪检查 |
| 交付与确认      | 完成系统部署、演示、文档提交、代码提交、用户验收和版本确认                  | 项目负责人、开发人员、测试人员、用户代表、指导教师或验收人员        | 可运行系统、交付文档、部署说明、验收材料、发布版本       | 验收测试、文档完整性检查、发布准入检查、用户确认    |
| 项目总结与过程改进  | 分析需求变更、进度偏差、团队协作、测试缺陷、质量问题和过程不足，提出改进措施         | 项目负责人、全体成员、质量负责人、指导教师或客户代表            | 项目总结、过程度量结果、缺陷统计、改进建议           | 复盘会议、度量分析、缺陷归因、改进建议评审       |

2.2 原有过程痛点分析

| 阶段         | 原有过程痛点                         | 产生原因                                            | 对效率或质量的影响                          |
|------------|--------------------------------|-------------------------------------------------|------------------------------------|
| 项目启动与计划    | 风险识别和任务估算主要依赖人工经验              | 项目早期信息不完整，团队对技术难度、需求变化和资源约束的判断容易不充分             | 计划偏差会导致后续迭代延期，风险遗漏会增加返工概率          |
| 需求获取与分析    | 需求整理耗时，需求表达容易不统一，需求追踪矩阵维护成本较高  | 需求来源分散，用户表达往往不够结构化，功能需求、非功能需求和验收标准之间需要反复对齐      | 需求遗漏或歧义会影响设计、编码、测试和验收，降低交付物一致性     |
| 原型设计与可行性验证 | 原型验证结论容易分散，准入标准不够明确            | 原型阶段强调快速试错，验证记录、用户反馈和技术限制如果没有及时沉淀，后续难以复用        | 后续设计和编码可能重复探索已有结论，技术风险可能延迟暴露       |
| 系统架构与详细设计  | UML建模、接口说明和设计文档编写成本较高，设计方案比较不足 | 系统通常包含多个模块、接口、数据库和外部依赖，设计人员需要在功能、性能、安全和可维护性之间权衡 | 模块边界不清或接口约定不完整会增加联调成本，并导致编码阶段理解偏差  |
| 编码实现与单元验证  | 样板代码多，代码审查依赖人工，重构和接口适配成本较高     | 业务系统中常见数据访问、接口封装、表单校验、异常处理和测试桩等重复结构             | 开发时间被重复实现占用，人工审查容易遗漏安全、异常处理和可维护性问题 |
| 集成测试与迭代修正  | 测试用例编写耗时，边界场景容易遗漏，需求覆盖率检查依赖人工  | 系统存在多模块协作、异常输入、权限差异、并发访问、外部服务失败等组合场景            | 缺陷可能在后期集成或验收阶段集中暴露，增加修正成本和交付风险     |
| 交付与确认      | 交付材料整理和验收说明依赖人工汇总              | 交付物包括系统版本、代码、文档、测试记录、部署说明和演示材料，来源较分散            | 文档与实际系统功能可能不一致，影响验收表达、版本确认和交付效率    |
| 项目总结与过程改进  | 日志分析、缺陷归因和复盘总结依赖人工，过程改进依据不足    | 缺陷记录、运行日志、测试报告和进度记录分散，缺少自动化归纳和度量分析              | 改进建议容易停留在经验描述，难以形成可复用的过程改进措施       |

三、AI软件工程应用趋势与匹配原则

近几年，生成式 AI 在软件工程中的作用已经不再局限于代码补全或函数生成，而是逐步进入需求整理、设计建模、编码实现、测试验证、运维分析等多个生命周期活动。对软件过程而言，这意味着 AI 的价值不只是提高某个开发人员的编码速度，而是可能影响需求文档、设计模型、代码质量、测试覆盖率和问题复盘等一系列过程交付物的形成方

式。

从行业实践看，AI 辅助软件开发已经具有较高普及度。Capgemini Research Institute 在 2024 年关于生成式 AI 软件工程的调研中提到，已经采用生成式 AI 的软件组织在软件生命周期整体生产率上观察到 7%—18% 的提升，其应用范围覆盖文档编写、编码、测试和质量改进等活动[1]。Stack Overflow 2025 开发者调查也从开发者侧面反映了这一趋势：84% 的受访者正在使用或计划使用 AI 工具，专业开发者中有 51% 每天使用 AI 工具[2]。这些数据说明，AI 工具已经从少数开发者的辅助尝试，逐渐变成软件工程过程改进中需要认真考虑的技术因素。

不过，AI 的引入并不等同于简单提高效率。Google Cloud DORA 2024 报告对 AI 影响的分析较为谨慎：AI 可以提升开发者的个体生产率、工作流畅感和满意度，但在交付吞吐、稳定性等组织级结果上也可能带来负面影响[3]。这说明，如果缺少清晰的质量控制、测试机制和过程约束，AI 生成的内容可能会放大返工、缺陷和协作成本。因此，在软件过程中引入 AI，不能只关注“生成得更快”，还必须关注“生成结果是否可靠、是否可追踪、是否经过评审”。

从具体软件工程活动看，需求、设计、编码、测试等阶段的 AI 应用成熟度并不完全相同。需求工程相关综述显示，生成式 AI 已被用于需求获取、需求分析、需求文档生成和需求验证等任务，但幻觉、可复现性不足和可解释性不足仍然是重要问题[4]。在软件架构领域，GenAI 能够辅助架构决策、需求到架构转换和架构重构，但其生成结果仍面临模型精度、隐私保护和评估标准不足等挑战[5]。因此，AI 更适合作为软件过程中的辅助角色，而不是直接替代需求分析师、架构设计人员、开发人员或测试人员。

基于以上调研，本报告将 AI 技术的定位概括为“过程增强工具”。也就是说，AI 主要承担资料整理、初稿生成、结构转换、模式识别和结果摘要等工作；项目成员则继续负责业务判断、技术取舍、质量标准制定、评审确认和版本控制。只有当 AI 输出物能够被人工检查、能够追踪到原始需求或代码依据，并且能够通过测试、评审或度量指标验证时，才适合被纳入正式软件过程。

结合原有软件过程阶段，本报告采用以下匹配原则：

| 匹配原则    | 在软件过程中的含义                                   |
|---------|---------------------------------------------|
| 对应过程痛点  | 每个AI场景必须对应原有软件过程中的某个低效活动节点，避免为了使用 AI 而使用 AI |
| 保留角色职责  | AI 只承担辅助性工作，不能替代项目负责人、设计人员、开发人员和测试人员的最终判断   |
| 交付物可检查  | AI生成内容必须能形成需求文档、UML模型、代码、测试用例或日志分析报告等可评审交付物 |
| 质量标准明确  | AI输出需要经过需求追踪、代码审查、单元测试、覆盖率检查或缺陷闭环验证         |
| 风险控制可执行 | 对幻觉、安全泄露、错误代码和错误根因分析设置人工确认和准入标准             |

四、AI技术应用场景匹配矩阵

| 原有过程阶段    | 原有痛点                     | 推荐AI应用场景           | AI可承担的工作                                             | 人工保留的职责                         | 预期改进效果                              |
|-----------|--------------------------|--------------------|------------------------------------------------------|---------------------------------|-------------------------------------|
| 需求获取与分析   | 需求整理耗时，表达不统一，需求追踪矩阵维护成本高 | AI辅助需求规格说明书与用户故事生成 | 根据原始需求材料生成SRS初稿、用户故事、验收标准，提示需求歧义并辅助建立需求追踪关系          | 判断需求是否真实存在，确认优先级，评审并形成需求基线      | 缩短需求文档初稿编写时间，提升需求表达一致性，增强需求到测试的追踪能力 |
| 系统架构与详细设计 | UML建模成本高，模块边界和接口方案比较不足   | AI辅助UML模型与系统设计草案生成 | 根据需求文档和原型结果生成模块划分、Mermaid/PlantUML类图、时序图、接口说明和备选架构方案 | 确认架构可实现性，评审模块边界、接口协议、数据库结构和部署约束 | 减少设计文档起草成本，促进多方案比较，降低联调阶段理解偏差       |
| 编码实现与单元验证 | 样板代码多，代码审查依赖人工，重构成本高     | AI辅助代码生成、重构与代码审查   | 生成低风险样板代码、接口适配代码、单元测试桩，提出重构和代码审查建议                   | 阅读并修改代码，执行版本控制、静态检查、单元测试和合并确认   | 降低重复编码成本，提高代码审查覆盖面，但保持人工对关键逻辑的控制    |
| 集成测试与迭代修正 | 测试用例编写耗时，边界场景和异常场景容易遗漏   | AI生成测试用例与自动化测试脚本   | 根据需求、接口和代码生成测试用例表、边界数据、自动化测试脚本和失败结果摘要                | 审核测试用例有效性，补充业务边界，运行测试并确认缺陷闭环    | 提高需求覆盖率和边界场景覆盖率，降低测试设计耗时，便于过程度量     |

| 原有过程阶段    | 原有痛点                 | 推荐AI应用场景         | AI可承担的工作                                   | 人工保留的职责                     | 预期改进效果                       |
|-----------|----------------------|------------------|--------------------------------------------|-----------------------------|------------------------------|
| 项目总结与过程改进 | 日志分析和缺陷归因依赖人工，复盘依据不足 | AI日志分析、异常检测与根因总结 | 对运行日志、测试失败记录 and 缺陷记录进行分类、摘要、可能原因排序和改进建议生成 | 验证根因，确认修复方案，形成正式缺陷记录和过程改进建议 | 缩短故障定位时间，使复盘从经验描述转向基于数据的过程改进 |

## 五、关键应用场景分析

### 5.1 AI辅助需求规格说明书与用户故事生成

在需求获取与分析阶段，团队通常需要将用户访谈、业务描述、功能设想和约束条件整理为需求规格说明书、用户故事、用例说明和需求跟踪矩阵。这个阶段的难点不在于单纯记录需求，而在于把零散、口语化、可能存在冲突的描述转化为结构化、可评审、可追踪的需求条目。对于多数软件项目来说，需求复杂度往往来自多方干系人输入、业务规则变化、边界条件不完整以及非功能需求难以量化等问题。因此，需求整理、表达规范化和需求追踪矩阵维护往往会消耗较多时间。

在这一环节中，AI比较适合作为需求分析人员的辅助整理工具。团队可以先提供访谈记录、业务流程说明或功能清单，再由AI生成需求规格说明书初稿、用户故事、验收标准和可能的异常场景。例如，对于“用户需要提交申请，审核人员需要查看申请并给出审批结果”这一类业务描述，AI可以进一步拆分为申请提交、材料校验、审核处理、结果反馈、异常退回等需求点，并补充对应的验收标准和测试关注点。

不过，AI生成的需求内容不能直接作为正式需求基线。需求是否真实存在、优先级是否合理、业务规则是否准确，仍需要由需求分析人员、项目负责人和相关干系人确认。比较可行的做法是将AI输出作为需求评审材料，对每条需求标注“采纳、修改、删除、待澄清”等状态，再将确认后的内容纳入需求规格说明书和需求跟踪矩阵。

从过程改进角度看，该场景并不是让AI替代需求分析，而是把原来完全依赖人工完成的“需求整理、SRS初稿编写、追踪关系维护”前置为“AI生成结构化初稿，人工进行判断和确认”。这样可以减少文档起草时间，提高需求表达的一致性，也有助于更早发现模糊需求和遗漏场景。考虑到需求工程研究中已经指出生成式AI存在幻觉、可复现性不足等问题[4]，本阶段必须设置明确的质量标准，包括需求来源可追溯、需求编号完整、验收标准明确、人工确认记录完整等。

## 5.2 AI辅助UML模型与系统设计草案生成

系统架构与详细设计阶段的主要任务，是将已经确认的需求进一步转化为系统结构、模块划分、接口协议、数据模型和部署方案。该阶段通常需要输出系统设计说明书、UML模型、接口文档和数据库设计文档。对于一般软件项目而言，系统往往同时包含前端、后端、数据库、第三方服务、权限控制和部署环境等组成部分，各模块之间存在数据依赖、调用依赖和时序依赖。设计人员如果完全从需求文档开始手工绘制UML图、整理接口说明，不仅耗时较长，也容易在模块边界、接口细节或异常流程上出现遗漏。

AI在设计阶段的价值主要体现在“生成可讨论的设计草案”。例如，团队可以将需求说明和业务流程输入AI，让其生成初步的模块划分、类图、时序图、活动图、接口清单或备选架构方案。特别是在使用Mermaid、PlantUML等文本化建模工具时，AI能够把自然语言需求转换为可编辑的模型代码，便于团队快速修改和评审。

需要注意的是，设计阶段的AI输出更适合作为讨论材料，而不是最终设计结论。软件架构相关研究已经表明，GenAI可以用于架构决策支持、需求到架构转换和架构重构辅助，但其输出仍然存在模型精度、隐私保护和评估标准不足等问题[5]。因此，在正式进入后续编码阶段之前，设计人员仍需要检查模块划分是否合理、接口定义是否完整、数据库设计是否满足业务约束、关键流程是否经过原型验证。

为了保证AI设计输出可控，本阶段可以设置设计准入标准：每个模块应能追溯到具体需求编号；每个接口应包含输入、输出、异常处理和调用方；每张UML图都应经过人工评审；涉及核心业务规则、外部服务、性能瓶颈或安全约束的设计，应由技术负责人进行确认。由此可见，AI改变的是设计材料的起草方式和模型转换效率，而不是替代架构设计人员进行最终技术决策。

## 5.3 AI辅助代码生成、重构与代码审查

编码实现与单元验证阶段直接影响系统能否按设计落地。该阶段通常包括模块代码实现、接口适配、数据处理、业务逻辑开发、单元测试编写和代码提交，交付物主要是源代码、单元测试报告和代码审查记录。在实际开发中，开发人员经常需要编写大量重复性代码，例如数据库增删改查、接口参数校验、数据格式转换、异常处理模板和测试桩代码。这些内容技术难度不一定高，但数量较多，容易占用开发时间。

AI编程工具在这一阶段的应用已经比较成熟。GitHub关于Copilot代码质量的研究显示，在特定实验任务中，使用Copilot的开发者通过全部单元测试的可能性更高，代码可读性和可维护性评价也有所提升[6]。GitHub在2025年推出Copilot异步编码Agent，也说明AI编码工具正在从简单的补全助手向更主动的开发协作者发展[7]。

在通用软件项目中，AI适合处理边界清晰、风险较低、验证方式明确的编码任务，例如生成CRUD代码、接口适配代码、参数校验逻辑、单元测试桩、错误处理模板和局部重构建议。对于核心业务规则、权限控制、支付结算、数据安全、并发控制等关键逻辑，AI只能提供参考实现或审查建议，开发人员必须理解代码含义，并根据项目约束进行修改。

比较稳妥的过程是：开发人员先提供需求编号、接口约束和必要的代码上下文，由AI生成局部代码或审查建议；随后开发人员阅读、修改并补充必要注释；再通过静态检查、单元测试和人工代码审查；最后才允许进入版本控制并合并到主分支。该场景可以记录AI代码采纳率、单元测试通过率、代码审查问题数、缺陷修复时间和返工次数等指标。这样既能利用AI降低重复编码成本，又能避免AI生成代码未经理解和验证就进入正式系统。

#### 5.4 AI生成测试用例与自动化测试脚本

在本报告所分析的几个场景中，测试验证阶段最适合作为后续原型验证对象。原因在于测试活动的输入和输出相对明确：输入可以是需求文档、接口说明或代码片段，输出可以是测试用例表、边界数据、自动化测试脚本和测试结果分析。与需求和设计相比，测试结果也更容易通过覆盖率、缺陷发现数、执行结果和人工评审进行量化评价。

原有集成测试与迭代修正阶段通常包括模块集成、接口联调、功能测试、异常测试、性能测试和需求跟踪检查，交付物包括集成测试记录、缺陷记录和修正结果。多数软件系统都存在大量容易被遗漏的边界场景，例如异常输入、重复提交、权限不足、网络延迟、外部服务失败、数据状态不一致、并发访问和配置错误等。如果完全依靠人工从头编写测试用例，测试覆盖范围往往受经验和时间限制影响较大。

AI可以在该阶段根据需求、接口和代码生成测试用例初稿，补充正常路径、边界路径和异常路径，也可以生成部分自动化测试脚本，并对失败结果进行摘要。AI软件测试研究指出，AI增强测试自动化正在围绕测试设计、测试执行、维护和评估形成新的分类与方法[8]。因此，将AI用于测试用例生成，不仅符合当前研究趋势，也比较适合课程作业中的原型验证要求。

在具体流程上，测试人员可以先向AI提供需求编号、接口说明和业务规则，由AI生成测试用例表和测试数据；随后测试人员根据需求跟踪矩阵检查覆盖情况，删除重复用例，补充关键业务场景，并确认自动化脚本是否可运行。为了避免“用例数量多但质量不高”的问题，应设置测试准入标准：每条核心需求至少对应一个正常用例和一个异常用例；关键业务流程、权限路径和异常路径必须覆盖；自动化脚本必须能够执行；缺陷记录应包含问题描述、责任人、修复状态和关闭结果。该场景的效果可以通过测试用例生成耗时、需求覆盖率、分支覆盖率、缺陷发现数和重复用例比例等指标衡量。

#### 5.5 AI日志分析、异常检测与根因总结

对于课程项目或小型软件项目而言，长期运维活动通常会被适当简化，但部署验证、基础问题修复和项目总结改进仍然不可缺少。系统在测试、部署或演示过程中，仍可能出现接口异常、外部服务失败、数据状态不同步、配置错误、权限异常和响应延迟等问题。如果仅依靠人工逐条查看日志和缺陷记录，不仅排查效率较低，项目总结中的改进建议也容易停留在经验描述层面，缺少数据支撑。

AIOps和根因分析相关研究表明，复杂系统中的根因分析通常会结合指标、链路追踪、日志和多模态数据，并逐渐与AI和自动化技术结合[9]。对于一般软件项目来说，没有必要搭建完整的AIOps平台，但可以采用轻量化方式引入AI日志分析。例如，团队可以统一日志记录格式，保留时间、模块、接口、错误码、异常堆栈、请求编号和用户操作等字

段；当测试或演示过程中出现异常时，将相关日志和缺陷记录输入AI，由其生成异常摘要、影响范围、可能原因排序和排查建议。

这一场景的关键是明确AI输出的性质：AI给出的不是最终根因结论，而是辅助排查线索。开发人员仍需要根据日志证据复现问题、验证原因、确认修复方案，并将最终结果写入缺陷记录和项目总结。相较于完全人工排查，AI可以帮助团队更快聚焦可疑模块和异常模式，也能让复盘内容更加结构化。

本阶段的质量标准应包括：日志来源明确、分析结论有对应证据、根因判断可以复现、修复结果可以验证、改进建议具有可执行性。通过这种方式，AI可以把项目总结与过程改进从“主观经验回顾”推进到“基于日志和缺陷数据的复盘分析”，提高后续过程优化的依据性。

## 六、风险与控制措施

| 风险    | 表现                             | 控制措施                                                         |
|-------|--------------------------------|--------------------------------------------------------------|
| AI幻觉  | 编造不存在的需求、接口、API或系统行为           | 所有AI输出必须标注依据来源，并通过需求评审、设计评审或代码审查确认                           |
| 质量不可控 | 生成内容表面完整但遗漏关键业务规则              | 建立准入标准，如需求追踪、UML评审、单元测试通过、覆盖率检查和缺陷闭环                         |
| 安全泄露  | 在提示词、代码或配置中暴露API Key、用户数据或内部配置 | 使用脱敏数据，禁止提交密钥，增加密钥扫描和权限检查。GitGuardian报告也提示AI服务相关密钥泄露增长明显[10] |
| 过度依赖  | 成员不理解AI生成的设计、代码或测试             | 要求提交人工修改记录、代码解释和评审意见，关键模块必须由负责人确认                            |
| 版本混乱  | AI多次生成内容导致文档和代码版本不一致           | 将AI生成物纳入版本控制，记录生成时间、输入材料、采纳状态和修改人                            |
| 评价困难  | 只描述效率提升，缺少量化证据                 | 设置过程度量指标，如耗时、覆盖率、缺陷数、审查问题数、采纳率和返工次数                          |

## 七、结论

通过对原有软件过程阶段、活动节点、角色职责和交付物的梳理可以看出，AI技术更适合嵌入那些输入输出相对明确、重复性较强、能够形成可检查交付物的过程环节。对于需求分析、系统设计、编码实现、测试验证和过程改进等活动，AI可以在文档生成、结构化整理、模型转换、代码辅助、测试设计和日志分析方面发挥作用；但在需求确认、架构取舍、核心代码理解、缺陷判定和版本准入等关键决策上，仍必须保留人工评审和责任确认。

本报告最终确定了五类适合引入 AI 的应用场景：AI 辅助需求规格说明书与用户故事生成、AI 辅助 UML 模型与系统设计草案生成、AI 辅助代码生成与代码审查、AI 生成测试用例与自动化测试脚本、AI 日志分析与根因总结。从软件过程管理角度看，AI 的作用不应被理解为自动完成软件开发，而应被定位为过程增强手段。AI 可以承担重复性、结构化、初稿性和辅助分析类工作，帮助团队提高过程执行效率和交付物一致性；项目成员则需要继续负责需求基线确认、设计评审、代码审查、测试准入、风险控制和版本管理。只有将 AI 输出纳入明确的评审机制、质量标准和过程度量体系，才能避免因幻觉、错误代码、安全泄露或过度依赖带来的新风险。

## 参考文献

---

[1] Capgemini Research Institute. *Turbocharging software with Gen AI: How organizations can realize the full potential of generative AI for software engineering*. 2024. <https://www.capgemini.com/insights/expert-perspectives/boosting-productivity-in-software-engineering-with-generative-ai>

[2] Stack Overflow. *2025 Developer Survey: AI*. 2025. <https://survey.stackoverflow.co/2025/ai>

[3] Google Cloud DORA. *Accelerate State of DevOps Report 2024*. 2024. <https://dora.dev/research/2024/dora-report/>

[4] Cheng H., Husen J. H., Lu Y., Racharak T., Yoshioka N., Ubayashi N., Washizaki H. *Generative AI for Requirements Engineering: A Systematic Literature Review*. arXiv, 2024/2025. <https://arxiv.org/abs/2409.06741>

[5] Esposito M., Li X., Moreschini S., Ahmad N., Cerny T., Vaidhyanathan K., Lenarduzzi V., Taibi D. *Generative AI for Software Architecture: Applications, Trends, Challenges, and Future Directions*. arXiv, 2025. <https://arxiv.org/abs/2503.13310>

[6] GitHub Blog. *Does GitHub Copilot improve code quality? Here's what the data says*. 2024. <https://github.blog/news-insights/research/does-github-copilot-improve-code-quality-heres-what-the-data-says/>

[7] GitHub Newsroom. *GitHub Introduces Coding Agent For GitHub Copilot*. 2025. <https://github.com/newsroom/press-releases/coding-agent-for-github-copilot>

[8] Schieferdecker I. K. *Navigating the growing field of research on AI for software testing: the taxonomy for AI-augmented software testing and an ontology-driven literature survey*. arXiv, 2025. <https://arxiv.org/abs/2506.14640>

[9] Wang T., Qi G. *A Comprehensive Survey on Root Cause Analysis in (Micro) Services: Methodologies, Challenges, and Trends*. arXiv, 2024. <https://arxiv.org/abs/2408.00803>

[10] GitGuardian. *The State of Secrets Sprawl 2026: AI-Service Leaks Surge 81% and 29M Secrets Hit Public GitHub*. 2026. <https://blog.gitguardian.com/the-state-of-secrets-sprawl-2026/>